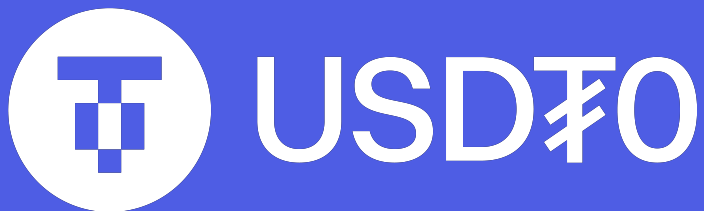


USDT0 Iota Package Security Audit



January 29, 2026

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Token Package	5
OFT Package	5
Security Model and Trust Assumptions	6
Privileged Roles	7
Low Severity	8
L-01 Rate Limiting Without OFT Fee Enables Low-Cost Denial of Service	8
Notes & Additional Information	8
N-01 Cross-Chain Transfers to Blacklisted Recipients Result in Permanent Token Loss	8
N-02 Unused Function	9
N-03 Code and Documentation Inconsistencies	9
N-04 Dust Forwarded to Fee Recipient During Debit Operations	10
N-05 Rate Limiter Decay Calculation Can Overflow Under Extreme Configurations	10
N-06 Missing Documentation	11
N-07 Fee Calculation Rounds Down in Favor of Users	11
Conclusion	13
Appendix	14
Issue Classification	14

Summary

Type	Token	Total Issues	8 (0 resolved)
Timeline	From 2026-01-12 To 2026-01-22	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (0 resolved)
		Notes & Additional Information	7 (0 resolved)

Scope

OpenZeppelin audited the [USDT0 package for Iota chain](#) repository at commit [c213287](#).

In scope were the following files:

```
├── oft
│   ├── sources
│   │   ├── codec
│   │   │   └── oft_msg_codec.move
│   │   ├── internal
│   │   │   ├── oft_fee.move
│   │   │   ├── pausable.move
│   │   │   └── rate_limiter.move
│   │   ├── oft-infos
│   │   │   └── oft_info_v1.move
│   │   ├── types
│   │   │   ├── oft_fee_detail.move
│   │   │   ├── oft_limit.move
│   │   │   ├── oft_receipt.move
│   │   │   ├── oft_send_context.move
│   │   │   ├── oft_sender.move
│   │   │   └── send_param.move
│   │   ├── oft.move
│   │   ├── oft_impl.move
│   │   └── oft_ptb_builder.move
├── token
│   └── sources
│       └── usdt0.move
```

System Overview

USDT0 IOTA contracts implement LayerZero's Omnichain Fungible Token (OFT) standard on the IOTA blockchain, enabling cross-chain transfers of USDT0 tokens. The system follows a lock-and-mint architecture where USDT is locked in an OFT Adapter on Ethereum, and equivalent USDT0 tokens are minted on destination chains maintaining 1:1 backing. The scope of the audit consists of two Move packages: a token package defining the USDT0 regulated currency, and an OFT package implementing the cross-chain transfer logic.

Token Package

The token package defines USDT0 as a regulated currency with 6 decimals using the `create_regulated_currency_v1` function from IOTA's `coin` module. This provides denylist functionality through a `DenyCap` and supports global pause capabilities. The `TreasuryCap` and `DenyCap` are transferred to the deployer, while the `metadata` is shared publicly.

OFT Package

The OFT package implements the core cross-chain transfer functionality through the `OFT` struct, which supports two treasury models via the `OFTTreasury` enum:

- **Standard OFT:** Mints and burns tokens using a `TreasuryCap` for cross-chain transfers.
- **OFT Adapter:** Escrows and releases existing tokens from a balance pool.

While Ethereum uses the **OFT Adapter** model, IOTA deployment of `USDT0` is expected to use **Standard OFT**.

The transfer flow works as follows:

- **Sending:** `send()` debits tokens (burn or escrow), applies fees, updates rate limiters and initiates the LayerZero message. `confirm_send()` then finalizes the send operation using a hot potato pattern via `OFTSendContext` to ensure atomicity.
- **Receiving:** `lz_receive()` or `lz_receive_with_compose()` decodes the OFT payload, updates rate limiters, credits tokens (mint or release), and transfers to the

recipient. While the `lz_receive()` function is used for basic transfer without any extra functionality, `lz_receive_with_compose()` can be used with an extra composed message that allows further cross-chain contract calls.

Additional features include the following:

- **Rate Limiting:** Sliding window rate limiters with linear decay for both inbound and outbound transfers. This is configurable per remote chain.
- **Fee Management:** Basis-point fees with default and per-destination rates. Fees are transferred to a configurable deposit address.
- **Pausable:** Emergency pause functionality via `Pausable` to halt all sending and receiving operations.
- **Migration:** Support for migrating to new OFT contracts via `MigrationCap`.

Non-Standard OFT Functions

Unlike the standard LayerZero OFT implementation, the `oft` contract includes `mint` and `redeem` functions that allow the admin to mint tokens directly to any address and burn tokens (should be sent from the admin). These functions operate independently of the cross-chain flow and are only available for the standard OFT treasury model (not the adapter). This allows the admin to adjust token supply outside of normal cross-chain operations. Direct token-metadata-updating functions for `name`, `symbol`, `description` and `icon URL` are also implemented on top of standard implementation.

Security Model and Trust Assumptions

The system's security relies on the correct operation of LayerZero's cross-chain messaging infrastructure, proper configuration by privileged roles, and adherence to the OFT standard's design constraints. The contracts assume external dependencies functions correctly and configured parameters are within safe bounds.

Throughout the in-scope codebase, the following trust assumptions were identified:

- The LayerZero protocol, including the endpoint, DVNs (Decentralized Verifier Networks), and executors, operates correctly and securely.

- Only one OFT Adapter exists across all chains for USDT0 and the remaining deployments use the Standard OFT model. Multiple adapters could cause race conditions and accounting inconsistencies.
- `shared_decimals` are configured such that the conversion from `shared_decimals` to `local_decimals` (`to_ld()`) does not overflow. Configuration should follow LayerZero's recommended decimal configurations and take IOTA specific conditions into account.
- Rate limit configurations across chains are coordinated. Mismatched limits (e.g., higher outbound on source than inbound on destination) can cause transfers to fail on the destination chain.
- Fee deposit addresses are configured before enabling fees. Otherwise, `apply_fee` will revert.
- The admin uses `mint` and `redeem` functions responsibly, as improper use could break the 1:1 backing invariant with the locked USDT on Ethereum.

Privileged Roles

The holder of the `AdminCap` role can perform the following privileged operations:

- Pause and unpause all OFT operations
- Set fee deposit address, default fees, and destination-specific fees
- Configure rate limits for inbound and outbound transfers per endpoint
- Mint tokens to any address and redeem (burn) tokens, bypassing the cross-chain flow
- Update token metadata (name, symbol, description, icon URL)
- Register the OApp with the LayerZero endpoint

The `MigrationCap` holder can migrate the OFT to a new contract, extracting the `CallCap` and treasury components.

Low Severity

L-01 Rate Limiting Without OFT Fee Enables Low-Cost Denial of Service

Both `send` and `lz_receive_internal` consume rate limit capacity regardless of whether an OFT fee is configured. When rate limiting is enabled on a pathway without fees, an attacker can exhaust the rate limit capacity at minimal cost via only paying gas and LayerZero messaging fees. This blocks legitimate users until the rate limit window resets.

Consider avoiding the use of rate limiting on pathways without OFT fees configured or clearly documenting this interaction.

Update: Acknowledged, not resolved. The team stated:

This behavior is an intentional product requirement in the current design. The associated risks will be documented. No change is planned for the current version.

Notes & Additional Information

N-01 Cross-Chain Transfers to Blacklisted Recipients Result in Permanent Token Loss

USDT0 is created as a `regulated currency` with denylist functionality via `DenyCap`. When tokens are sent cross-chain, they are burned or escrowed on the source chain before the message is delivered. If the recipient is blacklisted on IOTA, the `lz_receive` operation will fail when attempting to transfer the credited tokens. Since the blacklist status is unlikely to change, retries will continue to fail, leaving tokens permanently stuck.

Consider implementing a rescue mechanism that allows redirecting stuck tokens to an alternative address or clearly documenting this limitation for users and integrators.

Update: Acknowledged, not resolved. The team stated:

| This is a non-security issue.

N-02 Unused Function

The `new_unbounded_offt_limit` function from the `oft_limit` module has `public(package)` visibility but no other module from the package is using it.

Consider removing the unused function to improve the overall clarity and maintainability of the codebase.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

N-03 Code and Documentation Inconsistencies

Throughout the codebase, multiple instances of code and documentation inconsistencies were identified:

- In the `rate_limiter` module, the `unset_rate_limit_function` uses the `ESameValue` error code when no rate limit exists. In other modules' functions that check for the same behavior such as `oft_fee::unset_fee_bps`, the `ENotFound` error code is used instead which leads to an inconsistency.
- In the `oft_ptb_builder` module, all functions lack documentation for the `clock` parameter, whereas in other modules such as `oft`, `clock` is clearly documented in all functions that have it as a parameter.
- In the `oft_msg_codec` module, the `send_to_field` inside the `OFTMessage` struct is stored as an `address`-type value, whereas in the `encode` function, it is provided with a `Bytes32 data type`.

Consider addressing the above-listed inconsistencies to improve the clarity and maintainability of the codebase.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

N-04 Dust Forwarded to Fee Recipient During Debit Operations

In the `debit_view` function, the protocol removes "dust" that does not align with shared decimals. When no fee is configured, this [dust remains in the user's balance](#). However, if a [fee is active](#), the system calculates the fee percentage first and then removes the dust from the remaining amount. This sequence causes both the fee and the dust to be sent to the fee recipient. According to the [LayerZero documentation](#), dust is kept by the sender. However, the current implementation leads to an undocumented overcharge where the user loses a small amount of funds due to the dust removal.

Consider clearly documenting the fact that users will be charged a small extra amount corresponding to the dust amount whenever a fee is configured.

Update: Acknowledged, not resolved. The team stated:

This behavior is implemented to maintain consistency with EVM semantics. This is not a security issue.

N-05 Rate Limiter Decay Calculation Can Overflow Under Extreme Configurations

The `in_flight` function computes token decay as `(elapsed * limit) / window_seconds` in `u128`, then [casts the result to u64](#). If the intermediate value exceeds `u64::MAX`, the cast aborts, blocking all transfers on that pathway. Triggering this requires a combination of very long inactivity periods and misconfigured limits. While unlikely in practice, an admin misconfiguration could block cross-chain transfers after extended periods of no activity.

Consider capping the decay value to `u64::MAX` before casting, or returning zero early when elapsed time is sufficient to fully decay the in-flight amount.

Update: Acknowledged, not resolved. The team stated:

This scenario is highly unlikely in practice and requires extreme misconfiguration. Even in such cases, the configuration can be reset. No immediate fix is planned.

N-06 Missing Documentation

Throughout the codebase, multiple instances of missing documentation were identified:

- In the `usdt0::usdt0` module, the `USDT0_one-time-witness` lacks documentation.
- In the `usdt0::usdt0` module, the `init_function_constructor` lacks documentation.
- In the `oft_ptb_builder` module, the `OFTPtbBuilder_struct` lacks documentation for its usage.
- In the `oft_send_context` module, the `OFTSendContext_struct` lacks documentation for its inner elements.
- In the `oft` module, the `quote_oft_function` lacks documentation for the `clock` parameter.
- In the `oft` module, the non-standard OFT functions (`update_metadata_name`, `update_metadata_symbol`, `update_metadata_description`, and `update_metadata_icon_url`) lack documentation for their parameters.
- In the `oft` module, the non-standard OFT function `total_supply` lacks documentation.
- In the `rate_limiter` module, the `drop_function` lacks documentation.
- The `oft_msg_codec`, `oft_fee_detail`, `oft_limit`, `oft_receipt`, and `send_param` modules lack initial documentation.

Consider addressing the above instances of missing documentation to improve code readability and maintainability.

Update: Acknowledged, will resolve. The team stated:

The identified inconsistencies stem from the transitional nature of the implementation. Documentation clarifications will be added, and structural cleanup will be performed in the unified production release.

Regarding the `OFTMessage.send_to` field: it is typed as `address` from the receiver's perspective, while during `encode` it represents a destination-chain address and is therefore handled as `bytes32`.

N-07 Fee Calculation Rounds Down in Favor of Users

The `apply_fee` function calculates fees using integer division, which rounds down. This results in the protocol collecting slightly lesser fees than the exact basis point calculation would yield, with the difference favoring users.

Consider rounding up when calculating fees to favor the protocol or documenting the current behavior as intentional.

Update: *Acknowledged, will resolve. The team stated:*

The current rounding behavior is intentional and favors users. This will be explicitly documented.

Conclusion

The USDT0 IOTA contracts implement LayerZero's OFT standard on the IOTA blockchain, enabling cross-chain USDT/USDT0 transfers through a lock-and-mint architecture. The implementation closely follows LayerZero's reference design, with additions such as admin-controlled `mint` and `redeem` functions for direct token supply management, and direct metadata update capabilities.

The audit identified one low-severity and seven informational findings. The low-severity finding relates to an edge case in rate limiter configuration. The informational findings cover documentation inaccuracies, missing documentation, incorrect error usage, fee rounding behavior, and operational considerations around regulated currency interactions with cross-chain messaging. No critical- or high-severity vulnerabilities were found.

Overall, the codebase was found to be well-structured and to make good use of Move's type system for safety. Nonetheless, test coverage could be strengthened in several areas, such as migration functionality, compose flow happy paths, slippage handling, and integrated rate limiting tests. Expanding coverage in these areas would increase confidence in the system's overall robustness.

The Everdawn Labs team is appreciated for their prompt responses during the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.